Thesis Plan

Decentralized Multi-Agent Reinforcement Learning for Power Grid Topology Control

Corentin Plumet

July 3, 2026

Contents

Chapter 1

Thesis Plan

1.1 Plan Inspiration

This plan follows the structure of the reference thesis in `mAI2024deMolB.pdf`: start from the power-grid motivation and research questions, introduce the scientific background, describe the environment before the algorithms, then separate methods from results and close by answering the research questions. The adapted structure below is tuned to this repository: Grid2Op topology control, decentralized MAPPO, graph encoders, rerun ablations, and sparse intervention mechanisms.

1.2 Working Title

Decentralized Multi-Agent Reinforcement Learning for Sparse Power Grid Topology Control

1.3 Central Thesis Question

How can decentralized multi-agent reinforcement learning agents learn reliable power-grid topology control policies that preserve survival performance while scaling to combinatorial action spaces and avoiding unnecessary interventions?

1.4 Research Questions

1. How should the Grid2Op topology-control problem be formulated as a decentralized multi-agent decision problem?
2. Which architectural choices improve survival and generalization: flat MLP policies, graph neural network encoders, shared actor weights, critic design, or optimized critic updates?
3. How do initialization and training choices, especially the initial do-nothing bias and reward normalization, affect exploration and learned behavior?

4. Can sparse-control mechanisms, such as rho-based overrides, intervention gates, fixed penalties, or adaptive intervention budgets, reduce unnecessary topology actions without sacrificing episodic survival?

1.5 Expected Contributions

- A clear decentralized formulation of topology optimization in Grid2Op.
- A controlled comparison of MAPPO baselines, GNN encoders, critic variants, and initialization choices.
- An ablation study of sparse intervention mechanisms for more operator-like control.
- A reproducible experiment and analysis pipeline based on W&B histories, cached metrics, notebooks, and final evaluation scripts.

1.6 Chapter Outline

Chapter 1: Introduction

Goal. Motivate topology control as a low-cost but difficult operational tool for power-grid reliability.

Planned content.

- Energy-transition context: higher demand, renewable uncertainty, and stressed transmission networks.
- Topology control as an alternative or complement to redispatching and grid expansion.
- Why the problem is hard: sequential control, physical constraints, rare failures, and combinatorial topology actions.
- Why decentralized MARL is a natural fit: local substations, regional decision making, reduced per-agent action spaces, and scalability.
- Research questions, contributions, and thesis outline.

Chapter 2: Background and Related Work

Goal. Give the reader enough power-systems and RL background to understand the design choices.

Planned content.

- Power-grid operation: substations, busbars, line loading, overloads, contingencies, and remedial actions.
- Grid2Op and the Learning to Run a Power Network setting.
- Reinforcement learning basics: MDPs, policy gradients, PPO, and MAPPO.
- Multi-agent reinforcement learning: decentralized execution, centralized training, credit assignment, and non-stationarity.

- Graph neural networks for grid-structured observations.
- Safe, sparse, and constrained RL ideas relevant to intervention budgets.

Chapter 3: Environment and Problem Formulation

Goal. Define exactly what the agents observe, choose, optimize, and are evaluated on.

Planned content.

- Grid2Op environment used in the repository and the selected chronics.
- Multi-agent decomposition: agent-region or agent-substation assignment.
- Observation spaces: local grid state, line loadings, topology state, and optional graph observations.
- Action spaces: local topology actions, action 0 as do nothing, legal action constraints, and action-space reduction.
- Reward and evaluation metrics: episodic survival, illegal action rate, action-0 fraction, non-idle fraction, and train/test chronic split.
- Reproducibility setup: seeds, W&B logging, cached histories, and deterministic evaluation.

Chapter 4: Methods

Goal. Present the implemented learning systems and the ablations.

Planned content.

- MAPPO baseline: actor, critic, rollout collection, PPO objective, and centralized training assumptions.
- Baseline MLP agents and the A0 family of reruns.
- Graph-encoder agents: bus graph construction, GINE/GAT/GCN variants, node and edge pre-encoders, graph readout, and flat-feature concatenation.
- Critic variants: GNN critic versus MLP critic, legacy versus optimized critic updates.
- Initialization and exploration choices: entropy coefficient, entropy decay, and initial do-nothing probability.
- Sparse-control mechanisms: evaluation-time rho heuristic, intervention gate, fixed sparse-action penalties, and adaptive intervention budgets.
- Experimental protocol for ablations: controlled seeds, fixed evaluation cadence, and fair baseline comparisons.

Chapter 5: Experiments and Results

Goal. Show which design choices matter and what behaviors the agents learn.

Planned content.

- Main baseline performance: training stability and test survival.

- GNN architecture ablations: concat-flat, actor sharing, GINE versus GAT or weighted GCN, light versus full encoders, and critic design.
- A0 rerun ablations: optimized critic updates, smaller MLP, reward normalization, and do-nothing initialization bias.
- Sparse-control ablations: fixed penalty, global-safe weighting, local-safe adaptive budget, rho-threshold sensitivity, and budget target sensitivity.
- Behavioral analysis: how often agents act, whether interventions are local, and whether high survival comes with excessive topology churn.
- Generalization: seen versus unseen chronics and robustness across seeds.

Figures and tables to prepare.

- Table of run families and what each ablation changes.
- Survival curves for baseline, GNN, A0, and sparse-control comparisons.
- Action-frequency plots: action 0 fraction, any non-idle action, and multi-agent simultaneous interventions.
- Summary table with final survival, peak survival, and action sparsity.

Chapter 6: Discussion

Goal. Interpret the results rather than just restating them.

Planned content.

- Answer each research question directly.
- Explain which components appear essential, useful, or unnecessary.
- Discuss the tradeoff between survival and sparse interventions.
- Relate learned behavior to operator-like control: act rarely, act locally, and act when the grid is unsafe.
- Discuss limits: simulator scope, incomplete operational constraints, seed variance, training cost, and dependency on chosen chronics.

Chapter 7: Conclusion and Future Work

Goal. Close with the main thesis claim and next research directions.

Planned content.

- Short summary of the problem, approach, and core findings.
- Final conclusion about decentralized MARL for topology control.
- Future work: richer graph representations, larger grids, outage contingencies, constrained RL tuning, better coordination mechanisms, and operational validation.

1.7 Appendix Plan

- Appendix A: Environment details and action-space construction.
- Appendix B: Hyperparameters and training configuration tables.
- Appendix C: Full ablation run registry.
- Appendix D: Additional survival and behavior plots.
- Appendix E: Reproducibility notes for notebooks, cached W&B histories, and evaluation scripts.

1.8 Immediate Writing Order

1. Write Chapter 3 first, because it fixes notation and the experiment vocabulary.
2. Write Chapter 4 next, using the existing configuration READMEs as source notes for method variants.
3. Draft the Chapter 5 table of ablations before prose, so the results chapter has a stable spine.
4. Write Chapter 1 once the main conclusion is clearer.
5. Finish Chapters 6 and 7 after the final plots and summary metrics are frozen.

1.9 Open Decisions

- Confirm the final thesis title and whether to emphasize GNNs, sparse-control, or decentralized MARL most strongly.
- Decide which experiment families are final and which are only exploratory.
- Decide whether the main claim is performance improvement, improved sparsity at equal survival, or reproducible ablation insight.
- Choose the final bibliography style required by the university.

Chapter 2

Experimental Results

2.1 How to Read This Chapter

This chapter is organized as a sequence of experiment blocks. The goal is to make the thesis easy to navigate while many tests are still being added. Each block follows the same structure:

1. **Question.** What uncertainty or design choice does this test address?
2. **Baseline and change.** Which run is the control, and what is changed in the comparison runs?
3. **Protocol.** Which data split, checkpoint, seeds, and metric are used?
4. **Implementation detail.** Which algorithmic, architectural, or mathematical change is being tested, when this is relevant?
5. **Evidence.** Which table and figure summarize the result?
6. **Interpretation.** What can be concluded, and what still needs to be verified?

Using the same block structure for every new experiment should make it possible to add future MSc-thesis tests without rewriting the whole results chapter.

2.2 Evaluation Protocol

The experiments use a train/test chronic split. The training set contains 80% of the chronics and is used to update the policy. The test set contains the remaining 20% and is used to estimate generalization during training and for final evaluation.

During training, evaluation rollouts are periodically run on both the train and test splits using 10 episodes each time. In the current rerun notebook, the main reported metric is **test episodic survival**, expressed as a percentage. Higher survival means that the agent keeps the grid alive for a larger fraction of the evaluation episodes. The final plots show seed-aggregated mean curves, faint individual seed curves, and the variability band.

The final evaluation currently uses the saved checkpoint with the best combined train/test

survival. This selection rule should be reported explicitly because the test split participates in checkpoint selection; the values in this chapter are therefore best interpreted as controlled rerun comparisons, not as a final untouched test-set estimate.

2.3 Experiment Index

Table 2.1 gives a compact map of the experiment blocks included so far. New result sections should add one row to this index before adding detailed prose.

Table 2.1: Current result blocks and their role in the thesis.

Block	Main question	Primary evidence	Status
A0 core reruns	Which baseline training changes stabilize MAPPO?	Table 2.3, Figures 2.1 and 2.2	Drafted
Initial do-nothing bias	Does an initial action-0 prior help exploration?	Table 2.4, Figure 2.3	Drafted
GINE reruns	Do graph encoders improve survival over the A0 baseline?	Tables 2.5 and 2.6, Figure 2.4	Drafted
Heavy GINE	Does a larger GINE architecture help or overcomplicate optimization?	Table 2.7, Figure 2.5	Drafted
Sparse intervention control	Can decentralized agents intervene rarely, locally, and only when useful?	Tables 2.8 and 2.9	Design drafted

2.4 IEEE 14-Bus Experiments

The first result group focuses on the IEEE 14-bus topology-control task. The initial PPO/MAPPO baseline from the MARL2Grid framework was not satisfactory: the agents did not learn a robust policy and the survival rate remained low. The following experiments therefore test training and architecture changes that could make the baseline more stable.

2.4.1 A0 Core Reruns

Question. Which changes are needed to obtain a stable MLP MAPPO baseline on the 14-bus task?

Baseline and changes. The A0 family starts from the flat MLP MAPPO baseline and tests three main stabilization choices:

- increasing actor and critic capacity from two hidden layers to three hidden layers;
- using reward normalization;
- changing the critic update so that the critic is updated once for the whole batch instead of once per agent.

Implementation detail. There is no new mathematical objective in this ablation. The implementation changes are practical training choices: network depth, reward normalization, critic-update scheduling, and the initialization of the action-0 probability. Table 2.2 records these settings so that the survival plots can be interpreted without repeatedly restating the run names.

Table 2.2: A0-family run configurations. The reference run is `rerun_a0_opt`.

Run family	Actor/critic hidden layers	Reward norm.	Initial action-0 prob.	Optimized critic update	Difference from `rerun_a0_opt`
`rerun_a0_opt`	3×256 / 3×256	true	0.0	true	Reference baseline
`rerun_a0_nonopt`	3×256 / 3×256	true	0.0	false	Disables optimized critic updates
`rerun_a01`	2×256 / 2×256	true	0.0	true	Uses a smaller MLP actor and critic
`rerun_a02`	3×256 / 3×256	false	0.0	true	Disables reward normalization
`old_a0_nonopt_noval20`	3×256 / 3×256	true	0.3	false	Uses action-0 bias 0.3 and non-optimized critic updates
`rerun_bias_05`	3×256 / 3×256	true	0.5	true	Uses action-0 bias 0.5
`rerun_bias_07`	3×256 / 3×256	true	0.7	true	Uses action-0 bias 0.7

Evidence. Table 2.3 reports final and peak survival for the core A0 ablations. Figure 2.1 shows each comparison against the `rerun_a0_opt` baseline, and Figure 2.2 overlays all core A0 curves.

Table 2.3: A0 core rerun survival summary.

Run	Intended change	Final survival (%)	Peak survival (%)
`rerun_a0_opt`	A0 optimized baseline	93.4	98.6
`rerun_a0_nonopt`	Disable optimized critic update	93.1	95.8
`rerun_a01`	Smaller MLP actor/critic	93.7	100.0
`rerun_a02`	Disable reward normalization	81.8	97.1
`old_a0_nonopt_noval20`	Older non-optimized protocol	22.3	68.4

Interpretation. The optimized A0 baseline, the non-optimized critic-update variant, and the smaller MLP variant all finish around 93% survival. This suggests that the main A0 recipe is robust once the rerun protocol is fixed. Disabling reward normalization still reaches high peak survival, but it learns more slowly and finishes lower. The old no-validation, non-optimized baseline is clearly not competitive and should not be used as the main reference for later ablations.

2.4.2 Initial Do-Nothing Bias

Question. Does biasing the initial policy toward action 0, the do-nothing action, help or hurt learning?

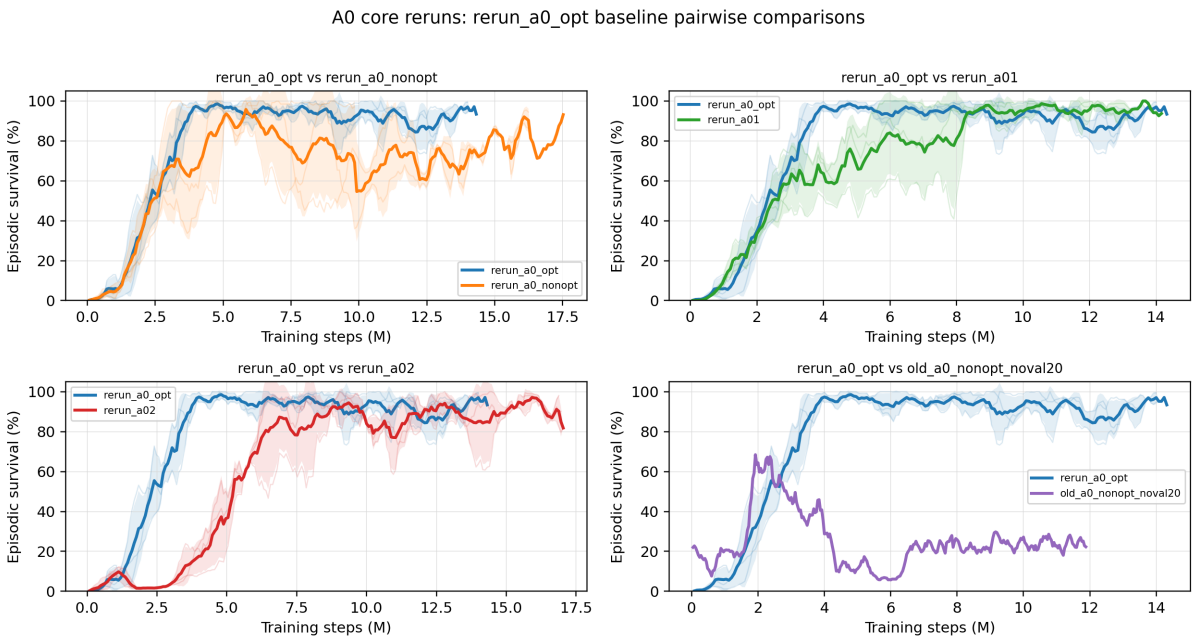

Figure 2.1: Pairwise A0 rerun comparisons against the `rerun_a0_opt` baseline.

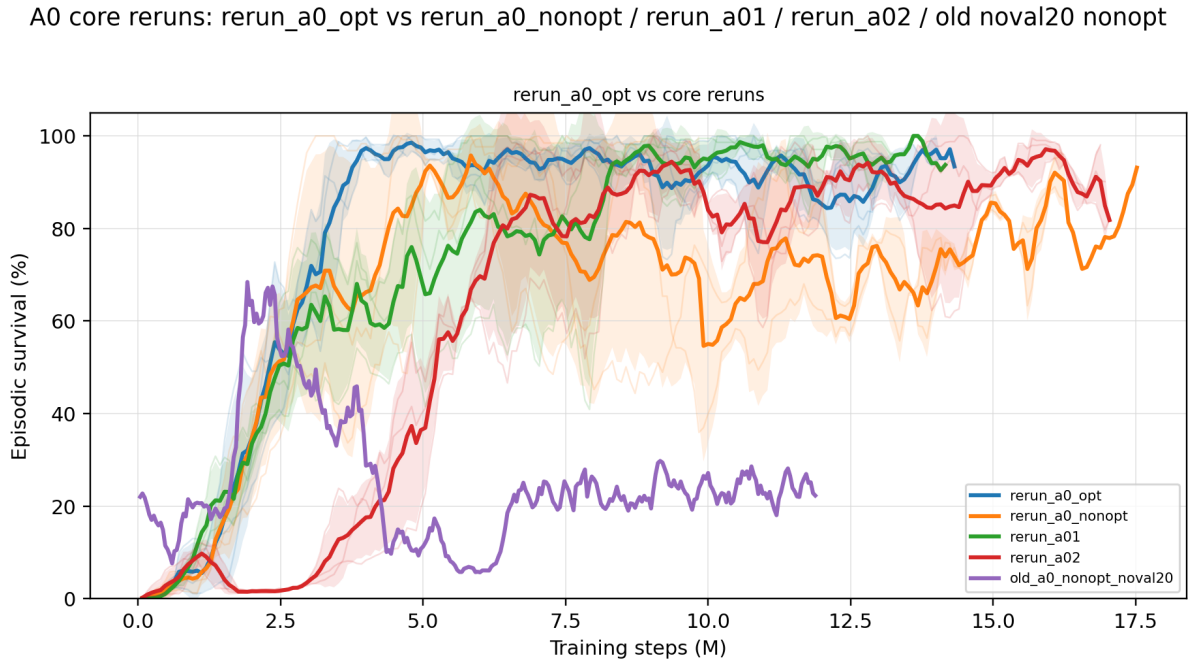

Figure 2.2: All A0 core rerun survival curves shown on one axis.

Baseline and changes. The A0 baseline uses no initial do-nothing bias in this rerun set. The bias controls keep the same general protocol but change the initial probability mass assigned to action 0.

Implementation detail. The bias is applied only at initialization: it changes the starting action distribution before learning, without changing the reward, architecture, or PPO objective. This makes the ablation a test of exploration bias rather than a test of representational capacity.

Evidence. Table 2.4 and Figure 2.3 compare the zero-bias A0 baseline with `rerun_bias_05` and `rerun_bias_07`.

Table 2.4: Initial do-nothing bias survival summary.

Run	Final survival (%)	Peak survival (%)
`rerun_a0_opt`	93.4	98.6
`rerun_bias_05`	50.8	70.2
`rerun_bias_07`	99.3	100.0

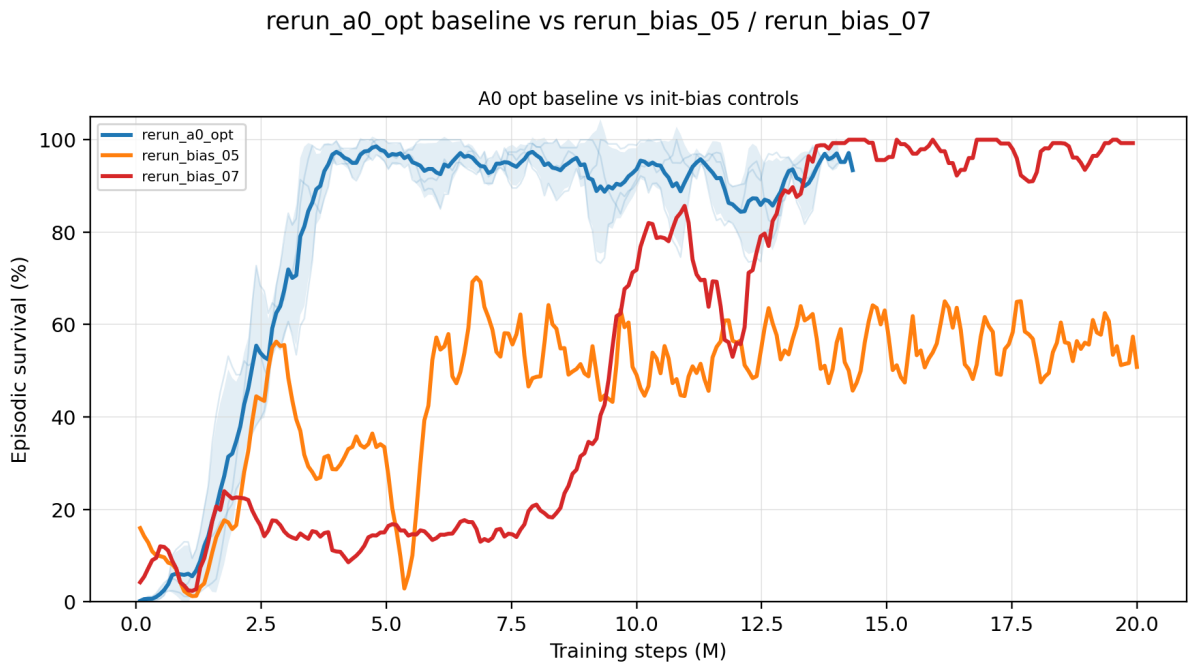

Figure 2.3: A0 baseline compared with initial do-nothing bias controls `rerun_bias_05` and `rerun_bias_07`.

Interpretation. The effect of the action-0 bias is large and non-monotonic. The 0.5-bias curve remains unstable and substantially below the zero-bias baseline, whereas the 0.7-bias curve eventually reaches the highest final survival in this comparison. The current conclusion is therefore not that larger bias is always better. Instead, the initialization prior appears to strongly shape exploration and should be treated as a first-order ablation variable.

2.4.3 GINE Encoder Reruns

Question. Can graph encoders improve topology-control performance, and which GINE design choices matter most?

Baseline and changes. The comparison starts from the historical `best_00` GINE baseline. The reruns then test lighter GINE encoders, MLP versus GNN critics, optimized critic updates, flat-feature concatenation, A0-style entropy, and different do-nothing-bias settings.

Protocol. All GINE runs use `gnn_type = gine`, reward normalization, 15,000,000 total timesteps, evaluation every 80,000 timesteps, 2,000 rollout steps, deterministic evaluation, and the same 80/20 chronic split used by the A0 experiments. The reference run is `best_00`.

Table 2.5: GINE rerun configuration summary. The reference run is `best_00`.

Run	Actor/critic encoder	GNN hidden/layers	Actor/critic heads	Concat flat	Edge pre-enc.	Entropy	Init action-0	Opt. critic	Difference from `best_00`
`best_00`	`gnn` / `gnn`	128 / 2	3×256 / 3×256	true	true	0.02	0.7	false	Baseline: heavy concat-flat GINE actor, GNN critic, and legacy critic updates
`best_10`	`gnn` / `mlp`	64 / 1	2×128 / 2×128	true	false	0.02	0.7	true	Light GINE actor, MLP critic, smaller heads, no edge pre-encoder, optimized critic updates
`best_11`	`gnn` / `gnn`	128 / 2	3×256 / 3×256	true	true	0.02	0.0	false	Bias ablation: same as baseline, but the initial do-nothing prior is 0.0
`best_12`	`gnn` / `gnn`	128 / 2	3×256 / 3×256	true	true	0.02	0.0	true	Bias 0.0 plus optimized critic updates; otherwise heavy concat-flat GINE with GNN critic
`best_13`	`gnn` / `gnn`	128 / 2	3×256 / 3×256	true	true	0.01	0.0	true	A0-style heavy GINE: bias 0.0, lower entropy 0.01, optimized critic updates
`best_14`	`gnn` / `mlp`	64 / 1	2×128 / 2×128	false	false	0.01	0.0	true	Light A0-style GINE: MLP critic, no concat-flat, lower entropy 0.01, bias 0.0, optimized critic updates

Implementation detail. The full GINE model architecture is described separately in Appendix A. This block separates representational changes from optimization changes. The GINE encoder changes how grid observations are embedded, while the critic type, flat-feature concatenation, entropy setting, and critic-update schedule affect how the PPO objective is optimized.

Evidence. Table 2.5 records the run configurations, Table 2.6 gives the selected survival values, and Figure 2.4 shows the pairwise survival curves against `best_00`.

Table 2.6: GINE rerun survival summary.

Run	Intended change	Final survival (%)	Peak survival (%)
`best_00`	Historical GINE baseline	59.2	93.9
`best_10`	Light GINE with MLP critic	78.9	90.4
`best_12`	Optimized critic and bias 0.0	82.5	85.7
`best_14`	Light A0-style GINE with MLP critic	95.7	97.4

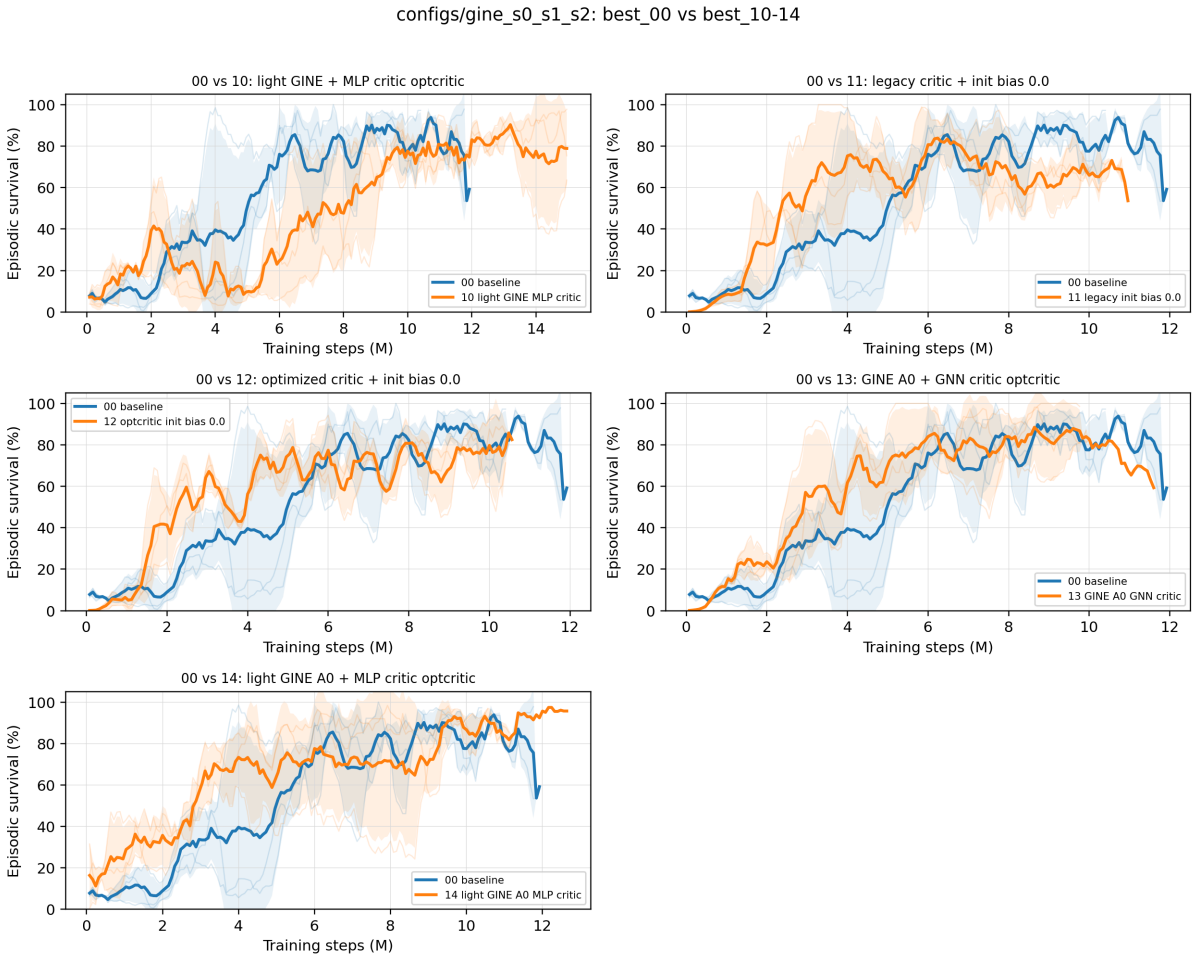

Figure 2.4: Seed-aggregated test episodic survival for GINE `best_00` against `best_10`–`best_14`.

Interpretation. The strongest GINE result is `best_14`, which reaches about 95.7% final survival. This suggests that the graph encoder can be competitive when it is paired with a simpler MLP critic and stable A0-style training choices. The heavier historical GINE baseline has high peak survival but much weaker final survival, which points to optimization stability rather than representational capacity as the main issue.

2.4.4 Heavy GINE

Question. Does the heavier concat-flat GINE baseline fail because it lacks capacity, or because specific architectural and optimization choices make it unstable?

Baseline and changes. The reference run is `best_00`, the historical shared-actor, concat-flat GINE with a GNN critic and legacy critic updates. The best-search 2 family compares this baseline against controlled alternatives that change the flat-feature concatenation, critic type, actor sharing, GNN size, entropy schedule, node identifiers, message-passing operator, and initialization bias.

Protocol. The configurations are read from `configs/gine_s0_s1_s2`. Each family is run with seeds `s0`, `s1`, and `s2`. Unless changed by the family, the runs use reward normalization, deterministic evaluation, 5 evaluation episodes, 15,000,000 total timesteps, evaluation every 80,000 timesteps, and the same 80/20 chronic split.

Implementation detail. Table 2.7 gives the parameter comparison used for this ablation. The table is based on the `s0` TOML file for each family; the corresponding `s1` and `s2` files keep the same configuration except for the training seed and run name.

Table 2.7: Heavy-GINE best-search 2 parameter comparison from `configs/gine_s0_s1_s2`.

Run	Main change from `best_00`	GNN	GNN size	Critic	Actor/critic heads	Concat	Shared	Edge pre.	Node ID	Entropy	Init action-0	Opt. critic
`best_000`	Removes flat-observation concatenation	`gine`	2×128	`gnn`	3×256 / 3×256	no	yes	yes	yes	0.02–0.02	0.7	no
`best_00`	Reference: heavy concat-flat GINE with GNN critic	`gine`	2×128	`gnn`	3×256 / 3×256	yes	yes	yes	yes	0.02–0.02	0.7	no
`best_01`	Enables optimized critic updates	`gine`	2×128	`gnn`	3×256 / 3×256	yes	yes	yes	yes	0.02–0.02	0.7	yes
`best_02`	Replaces the GNN critic with an MLP critic	`gine`	2×128	`mlp`	3×256 / 3×256	yes	yes	yes	yes	0.02–0.02	0.7	no
`best_03`	Uses non-shared actor GNN weights	`gine`	2×128	`gnn`	3×256 / 3×256	yes	no	yes	yes	0.02–0.02	0.7	no
`best_04`	Uses a lighter GINE encoder	`gine`	1×64	`gnn`	3×256 / 3×256	yes	yes	yes	yes	0.02–0.02	0.7	no
`best_05`	Decays entropy from 0.02 to 0.0	`gine`	2×128	`gnn`	3×256 / 3×256	yes	yes	yes	yes	0.02–0.00	0.7	no
`best_06`	Removes learned node-ID embeddings	`gine`	2×128	`gnn`	3×256 / 3×256	yes	yes	yes	no	0.02–0.02	0.7	no
`best_07`	Replaces GINE with GAT message passing	`gat`	2×128	`gnn`	3×256 / 3×256	yes	yes	yes	yes	0.02–0.02	0.7	no
`best_08`	Replaces GINE with weighted GCN message passing	`gcn`	2×128	`gnn`	3×256 / 3×256	yes	yes	yes	yes	0.02–0.02	0.7	no
`best_09`	Light no-concat GINE with MLP critic and optimized critic updates	`gine`	1×64	`mlp`	2×128 / 2×128	no	yes	no	yes	0.02–0.02	0.7	yes
`best_10`	Light concat-flat GINE with MLP critic and optimized critic updates	`gine`	1×64	`mlp`	2×128 / 2×128	yes	yes	no	yes	0.02–0.02	0.7	yes
`best_11`	Removes the initial do-nothing bias	`gine`	2×128	`gnn`	3×256 / 3×256	yes	yes	yes	yes	0.02–0.02	0.0	no

Evidence. Figure 2.5 compares the `best_00` baseline against each heavy-GINE alternative. Each panel shows the seed-averaged test-survival curve for `best_00` and one comparison family, with faint seed curves and variability bands.

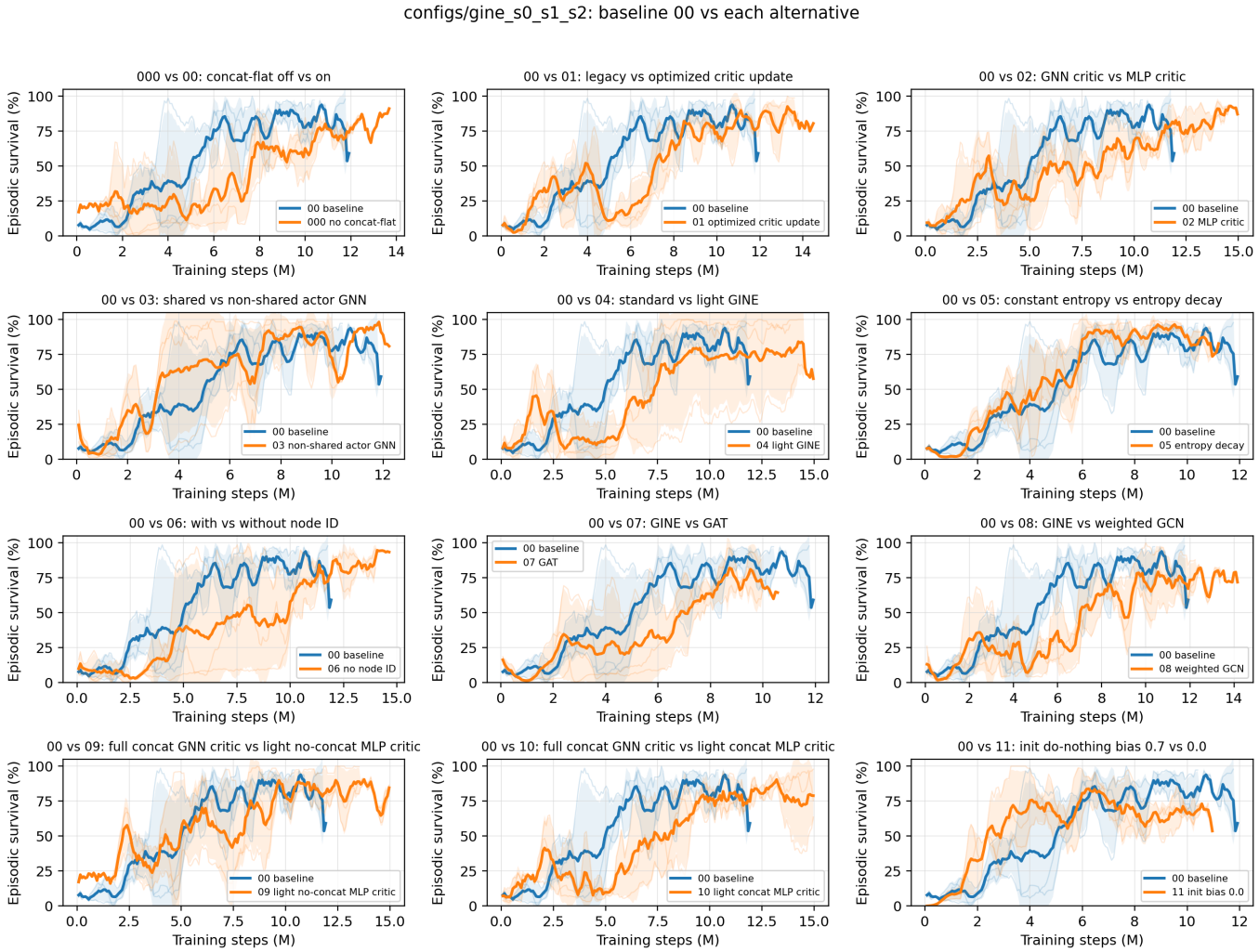

Figure 2.5: Heavy-GINE best-search 2 survival comparisons. Each subplot compares the `best_00` heavy concat-flat GINE baseline against one controlled alternative from `configs/gine_s0_s1_s2`.

Interpretation. The broad search suggests that the heavy baseline is not limited by capacity alone. Several simpler or more constrained variants improve final survival over `best_00`, especially removing node-ID embeddings, removing flat concatenation, using an MLP critic, and combining a light GINE actor with an MLP critic. In contrast, simply switching message passing to GAT or weighted GCN, or removing the initial do-nothing bias while keeping the heavy GINE/GNN-critic structure, does not clearly solve the stability problem.

2.5 Decentralized Sparse Intervention Control

Question. Can the agents keep high survival while behaving more like operators: acting rarely, acting locally, acting mainly when the grid state makes an intervention useful, and keeping the final policy decentralized?

Baseline and change. This block starts from the flat decentralized MAPPO topology-control actor. For agent i, the baseline policy is one categorical distribution over the full local action space:

$$\pi_i(a_i \mid o_i), \qquad a_i \in \{0, 1, \ldots, |\mathcal{A}_i| - 1\}.$$

Action 0 is the no-op action. During training, actions are sampled from the policy, while deterministic evaluation uses the greedy action by default:

$$a_{i,t}^{eval} = \arg\max_a \pi_i(a \mid o_{i,t}).$$

The sparse-control roadmap tests four ways to reduce unnecessary topology changes: evaluation-time rho overrides, an intervention-gated actor, fixed non-idle action penalties, and adaptive Lagrangian intervention budgets.

Motivation. The intervention gate is motivated by hierarchical topology-control work, where the action can be decomposed into "do nothing versus intervene", then where to act, then which local topology configuration to apply [1], [2]. In this thesis only the first level is adapted: each regional agent independently decides whether to do nothing or to execute one of its existing local topology actions. The fixed and adaptive sparse-control objectives are closer to constrained and budgeted RL: non-idle topology interventions are treated as a limited resource, following the general Lagrangian view of constrained policy optimization and budgeted control [3], [4], [5]. This is also operationally meaningful because switching frequency and topology deviation are real power-grid objectives, not just cosmetic metrics [6].

2.5.1 Evaluation-Time Rho Heuristics

Implementation detail. The rho heuristic is an evaluation-only diagnostic. It does not change the training objective and it does not prove that the policy learned sparse behavior. The policy first proposes the deterministic action $\bar{a}_{i,t}$, then the evaluator may replace it with action 0.

The global version uses the pre-action maximum line loading

$$\rho_t^{global} = \max_{\ell \in \mathcal{L}} \rho_{\ell,t},$$

and executes

$$a_{i,t}^{exec} = \begin{cases} 0, & \rho_t^{global} < \rho_{safe}, \\ \bar{a}_{i,t}, & \text{otherwise}, \end{cases} \qquad \rho_{safe} = 0.90.$$

The local version is decentralized. Agent i computes the maximum rho on the lines touching its regional substations,

$$\rho_{i,t}^{local} = \max_{\ell \in \mathcal{L}_i} \rho_{\ell,t},$$

and only that agent is forced to no-op when $\rho_{i,t}^{local} < \rho_{safe}$. A boundary line can appear in the local neighborhood of both adjacent agents, but no communication is required: both agents can observe the line through their own physical neighborhood.

Interpretation. The heuristic is useful to ask what happens if safe-state interventions are blocked, but it is not a learned method. Its action-0 rates must therefore be reported as final executed evaluation actions, not as evidence that the policy itself prefers action 0.

2.5.2 Intervention-Gated Actor

Implementation detail. The gated actor changes the actor factorization. Instead of one categorical head over all actions, each agent has a gate head and a non-idle action head:

$$\pi_i^{gate}(g_i \mid o_i), \qquad g_i \in \{0, 1\},$$

where 0 means do nothing and 1 means intervene, and

$$\pi_i^{nonidle}(b_i \mid o_i), \qquad b_i \in \{1, \ldots, |\mathcal{A}_i| - 1\}.$$

During training, the gate is sampled first. If $g_{i,t} = 0$, then $a_{i,t} = 0$. If $g_{i,t} = 1$, the final action is sampled from the non-idle head. The PPO log-probability of the executed action is therefore

$$\log P(a_i = 0) = \log P(g_i = 0),$$

and, for $a_i > 0$,

$$\log P(a_i) = \log P(g_i = 1) + \log P(b_i = a_i \mid g_i = 1).$$

Two deterministic decoders are used. The `final_action_map` decoder selects the most likely final executed action:

$$P(a_i = 0) = P(g_i = 0), \qquad P(a_i = a > 0) = P(g_i = 1) P(b_i = a \mid g_i = 1).$$

The `hierarchical_greedy` decoder first chooses the most likely gate decision, then chooses the most likely non-idle action only if the gate selects intervention.

The original coupled entropy term is

$$H_i = H(g_i) + P(g_i = 1)H(b_i).$$

This can accidentally reward intervention because larger $P(g_i = 1)$ unlocks more non-idle entropy. The separated entropy variant is

$$H_i = \alpha_{gate}H(g_i) + \alpha_{nonidle}H(b_i),$$

and is used in the gate-plus-budget diagnostic runs.

Interpretation. The gate is learned and decentralized, but it can restrict exploration more strongly than a reward penalty. If it collapses early to no-op, the non-idle head is rarely executed and receives little learning signal. For this reason, the gate is treated as an important diagnostic rather than the main sparse control contribution.

2.5.3 Fixed Sparse-Action Penalty

Implementation detail. The Sparse16 sweep adds a fixed reward penalty to the final executed action of each agent:

$$r'_{i,t} = r_{i,t} - \lambda_{fixed}\mathbf{1}[a_{i,t} \neq 0],$$

with

$$\lambda_{fixed} \in \{0.0, 0.001, 0.003, 0.01\}.$$

The grid is

$$\text{actor} \in \{\text{flat}, \text{gated}\}, \qquad \lambda_{fixed} \in \{0.0, 0.001, 0.003, 0.01\}, \qquad \text{seed} \in \{0, 1, 2\},$$

which gives $2 \times 4 \times 3 = 24$ runs. All Sparse16 runs set `safe_intervention_penalty = 0.0`, so the penalty is not rho-weighted.

Interpretation. Sparse16 changes only the training reward. It does not override actions during evaluation and it does not hard-mask exploration during training. The policy can still sample non-idle actions, but each such action must be useful enough to pay its fixed cost. The key comparison is `sparse16_flat_p010` versus `sparse16_gated_p010`, which asks whether the gate adds value once a simple sparse objective already exists.

2.5.4 Adaptive Intervention Budget

Implementation detail. The adaptive intervention budget turns sparse topology control into a local constraint. For each agent,

$$c_i(s_t, a_{i,t}) = \mathbf{1}[a_{i,t} \neq 0]w_i(s_t),$$

and the desired constraint is

$$\mathrm{E}_{\pi_i}\left[c_i(s_t, a_{i,t})\right] \leq d.$$

The implemented rollout reward removes the constant term that does not affect the PPO action gradient:

$$r'_{i,t} = r_{i,t} - \lambda_i c_i(s_t, a_{i,t}).$$

After each rollout, the local multiplier is updated by

$$\lambda_i \leftarrow \mathrm{clip}\left(\lambda_i + \eta(\hat{C}_i - d), 0, \lambda_{\max}\right), \qquad \hat{C}_i = \frac{1}{T}\sum_{t=1}^{T} c_i(s_t, a_{i,t}).$$

If the observed cost $\hat{C}_i$ is above the budget d, future interventions become more expensive; if it is below the budget, the penalty relaxes. The default settings are `intervention_budget_lr = 0.02`, `intervention_budget_init_lambda = 0.0`, and `intervention_budget_max_lambda = 10.0`.

Three cost modes are used:

- **Non-idle cost:** $w_i(s_t) = 1$, so $c_i(s_t, a_{i,t}) = \mathbf{1}[a_{i,t} \neq 0]$. This is an adaptive version of a plain sparse-action penalty.
- **Global-safe cost:** $w^{global}(s_t) = \sigma(k(\rho_{safe} - \rho_t^{global}))$. This penalizes non-idle actions mostly when the whole grid is safe.
- **Local-safe cost:** $w_i^{local}(s_t) = \sigma(k(\rho_{safe} - \rho_{i,t}^{local}))$. This is the decentralized version: safe local states make interventions expensive, while stressed local states make them cheap.

In the AIB setting, $\rho_{safe} = 0.90$ is not a hard action override. It is the center of a smooth sigmoid safety weight.

AIB experiment grid. The first AIB folder, `configs/adaptive_intervention_budget_7`, contains the main local-budget ablations:

- `aib_00_flat_local_t020`: flat actor, local-safe cost, $d = 0.20$;
- `aib_01_flat_local_t010`: stricter local-safe budget, $d = 0.10$;
- `aib_02_flat_local_t035`: looser local-safe budget, $d = 0.35$;
- `aib_03_gate_hgreedy_sep_local_t020`: gated actor, hierarchical-greedy evaluation, separated entropy, local-safe cost, $d = 0.20$;
- `aib_04_flat_nonidle_t020`: flat actor with adaptive plain non-idle cost, $d = 0.20$.

The comparison `aib_00_flat_local_t020` versus `aib_04_flat_nonidle_t020` isolates the value of local rho weighting.

The mechanism-ablation folder `configs/adaptive_intervention_budget_mechanism_15` contains 15 additional flat-actor runs. It separates fixed versus adaptive coefficients, plain non-idle versus state-dependent costs, global versus local rho weighting, and rho-threshold sensitivity.

Table 2.8: Sparse-control mechanisms and where they affect the policy.

Method	Training effect	Evaluation effect	Main risk
Baseline	Samples from the flat policy	Greedy argmax by default	Standard entropy-controlled exploration
Rho heuristic	No training change	Hard no-op override in safe states	Evaluation is manually changed
Intervention gate	Changes action factorization and sampling	Final-action MAP or hierarchical greedy	Gate collapse can starve non-idle exploration
Sparse16	Fixed reward penalty on non-idle actions	No action override	Penalty coefficient must be tuned
AIB non-idle	Adaptive reward penalty on non-idle actions	No action override	Multiplier can grow if target is too strict
AIB local-safe	Adaptive state-dependent reward penalty	No action override	Depends on the local rho safety-weight design

Table 2.9: Main sparse-control comparisons to report.

Question	Comparison
Does a fixed sparse penalty already work?	Baseline versus `sparse16_flat_p010`
Does the gate help beyond a sparse objective?	`sparse16_flat_p010` versus `sparse16_gated_p010`
Fixed versus adaptive coefficient?	`sparse16_flat_p010` or `aibm_00_fixed_nonidle_p006` versus `aib_04_flat_nonidle_t020`
Does local-safe weighting matter?	`aib_00_flat_local_t020` versus `aib_04_flat_nonidle_t020`
Does global-safe weighting matter?	`aibm_02_adaptive_global_t020` versus `aib_00_flat_local_t020`
Is $\rho_{safe} = 0.90$ special?	`aibm_03` with 0.85, `aib_00` with 0.90, and `aibm_04` with 0.95
Is the heuristic only an evaluation trick?	Rho-heuristic runs versus learned sparse/AIB runs

Metrics to report. The main performance metric remains episodic survival. It should be paired with sparsity and physical-diagnostic metrics: per-agent action-0 fractions, per-agent non-idle rates, the distribution of how many agents act simultaneously, intervention-budget multipliers, budget costs and violations, the rate of interventions in costly safe states, pre- and post-action max rho, and topology-distance changes.

Interpretation. The current roadmap interpretation is that the gate is not the strongest contribution by itself. Sparse reward shaping already improves action-0 usage, the adaptive budget makes the sparse penalty self-tuning, and the local-safe cost makes the penalty more physically meaningful. In particular, `sparse16_flat_p010` is a strong fixed-penalty baseline, while the most principled learned sparse-control candidate is `aib_00_flat_local_t020`: it keeps the original decentralized actor, does not rely on an evaluation-time override, learns the intervention penalty adaptively, and uses a local state-dependent safety weight.

2.6 Current Takeaway

The current reruns indicate that the strongest direction is not a single isolated modification. Stable performance comes from a combination of A0-style training choices, reward normalization, careful initialization, and architectures that keep the critic and graph encoder simple enough to optimize reliably. Optimized critic updates and lighter GINE/MLP-critic variants can improve the final GINE rerun performance, but the A0 baseline remains the clearest stable reference point for future ablations. The sparse-control roadmap adds a second axis to the thesis: after survival is stabilized, the next question is whether the same decentralized agents can achieve high survival with fewer unnecessary topology interventions.

Appendix A

GINE Architecture

This appendix will describe the GINE encoder used in the graph-based topology control experiments. The final version should specify the graph construction, node and edge features, message-passing layers, pooling/readout operation, and how the actor and critic heads consume the encoded graph representation.

Bibliography

[1] J. Manczak et al., *Hierarchical reinforcement learning for power network topology control*, 2023. arXiv: 2311.02129. [Online]. Available: https://arxiv.org/abs/2311.02129

[2] E. van der Sar et al., *Hierarchical multi-agent reinforcement learning for power grid topology optimization*, 2023. arXiv: 2310.02605. [Online]. Available: https://arxiv.org/abs/2310.02605

[3] J. Achiam, D. Held, A. Tamar, and P. Abbeel, "Constrained policy optimization," *International Conference on Machine Learning*, 2017. [Online]. Available: https://arxiv.org/abs/1705.10528

[4] A. Stooke, J. Achiam, and P. Abbeel, "Responsive safety in reinforcement learning by PID lagrangian methods," *International Conference on Machine Learning*, 2020. [Online]. Available: https://arxiv.org/abs/2007.03964

[5] N. Carrara, E. Leurent, R. Laroche, T. Urvoy, and O.-A. Maillard, "Budgeted reinforcement learning in continuous state space," *Advances in Neural Information Processing Systems*, 2019. [Online]. Available: https://arxiv.org/abs/1903.01004

[6] L. Lautenbacher et al., *Multi-objective reinforcement learning for power grid topology control*, 2025. arXiv: 2502.00040. [Online]. Available: https://arxiv.org/abs/2502.00040

[7] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[8] C. Yu et al., "The surprising effectiveness of ppo in cooperative multi-agent games," *Advances in Neural Information Processing Systems*, 2022.

[9] B. Donnot, "Grid2op: A testbed platform to model sequential decision making in power systems," *arXiv preprint arXiv:2011.07929*, 2020.

[10] A. Marot et al., "Learning to run a power network challenge for training topology controllers," *Electric Power Systems Research*, 2021.

[11] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," *International Conference on Learning Representations*, 2017.